

丹铅——基于大模型的辅助阅读网页项目报告

一、团队介绍

徐以鑫	远程云数据库搭建，RAG 技术实现，后端部分功能实现
肖子丘	市场竞品调查，后端具体功能实现，langchain 框架搭建
朱季阳	前端页面实现，后端框架搭建，ppt 等报告撰写

二、背景介绍：

随着大学学习的不断深入，论文阅读逐渐成为学术研究和知识积累的重要环节。然而，对于许多同学来说，论文阅读并非易事，尤其是对于刚刚接触学术研究或尚未完全掌握相关领域知识的同学而言，论文阅读往往伴随着诸多挑战。首先，国内许多同学在英文阅读理解方面存在一定困难，尤其是学术论文中使用的专业术语和复杂句式，与日常英语表达有较大差异，导致理解不够准确。其次，许多论文涉及的专业知识较为深奥，对于尚未系统学习相关领域知识的同学来说，理解起来尤为吃力。此外，由于同学们日常需要兼顾其他学业任务、社团活动或个人事务，很难抽出大量时间进行持续性的长时阅读，而论文通常篇幅较长且上下文联系紧密，一旦中断阅读，重新进入状态会耗费更多时间和精力。最后，论文中常常引用大量参考文献，这些文献与论文内容紧密相关，但阅读这些文献会进一步增加阅读的工作量，使得整体阅读过程更加繁重。

为了帮助同学们更高效地阅读论文，我们基于云服务和 LangChain 框架开发了一款辅助论文阅读的集成网页。通过前期调研，我们总结出同学们在论文阅读过程中常见的几大困难：

1.英文理解不够准确

论文正文部分的一些词汇可能与日常所用的意义不同，尤其是在特定学术领域中的术语和表达方式，理解起来较为吃力。例如，某些词汇在普通语境和学术语境下的含义可能完全不同，这会导致理解偏差或困惑。

2.专业知识难以理解

一些专业性极强的文章涉及深奥的理论、模型或实验方法，对于尚未系统学习相关知识的同学来说，阅读起来非常困难。例如，计算机科学领域的论文可能包含复杂的算法推导，而生物医学领域的论文可能涉及大量的专业术语和实验流程。

3.连续性阅读吃力

论文的上下文联系紧密，前后内容往往相互呼应，而同学们日常需要应对其他学业压力或事务，导致阅读过程容易中断。一旦中断，重新梳理上下文关系会耗费大量时间和精力，影响阅读效率。

4.难以充分利用参考文献

许多论文中的内容与参考文献紧密相关，而阅读这些参考文献会显著增加阅读的工作量。例如，某些论文的核心方法或实验结果可能需要参考多篇文献才能完全理解，这进一步加大了阅读的难度和时间成本。

三、市场调研：

为了深入了解目前市场上的竞品，我们进行了深入的调研

竞品	核心功能	优势	不足
Semantic Scholar	论文推荐、引用网络分析、主题分类	简洁的用户界面；强大的引用网络功能；免费使用	生成摘要质量有限，缺乏深度问答功能；不支持多语言翻译
Connected Papers	论文关联图谱展示，帮助用户探索相关研究	可视化关联网络，直观显示研究方向	功能单一，仅支持关联分析，不支持问答、摘要和翻译
Explainpaper	高亮论文内容并生成简单解释，支持PDF上传	针对性解释和问答功能；直接处理PDF文件	功能范围窄，仅适用于本地文件；解释质量依赖输入内容，缺乏上下文理解
Scite	论文引用上下文分析，显示正向、负向或中立引用	独特的引用情感分析功能，帮助用户快速判断论文价值	不支持全文摘要和翻译；使用场景局限于引用分析
ChatGPT (API 定制)	基于用户需求生成论文摘要、翻译和解释	强大的语言理解和生成能力；可扩展性强	针对论文的优化程度有限；需要订阅付费
Google Scholar	学术论文搜索、论文引用计数、相关论文推荐	广泛的数据库覆盖；搜索功能强大	不支持摘要生成、翻译和深度分析功能

功能	Semantic Scholar	Connected Papers	Explainpaper	Scite	ChatGPT (API 定制)	Google Scholar
论文摘要生成	✗	✗	✗	✗	✓	✗
多语言翻译支持	✗	✗	✗	✗	✓	✗
深度问答功能	✗	✗	✓	✗	✓	✗
关联网络分析	✗	✓	✗	✗	✗	✗
引用情感分析	✗	✗	✗	✓	✗	✗
数据库覆盖范围	高	中	低	中	高	高
价格	免费	免费	订阅制	订阅制	订阅制	免费

而对于目前国内大热的豆包、智谱等大模型而言，以豆包为例，没有搜集数据能力，场景分析能力差，许多专业性强的解释牵强附会，对于专业性要求极高的论文阅读方面是极其致命的，它们更多的是一个普适性的人工智能，而我们的项目致力于通过大语言模型赋能教育，专注于辅助阅读方向，从阅读前生成大致摘要到阅读中的智能笔记和关键词解释，再到阅读后的总结生成，我们希望不仅在论文阅读中，在之后的其他例如报刊、杂志、小说等阅读方面都能起到辅助的作用，真正促进进入全民智能阅读时代

四、设计思路：

由于时间和期末考试等多重因素的限制，我们选择以较为熟悉的网页作为项目初期开发的主要载体。这种选择不仅能够加快开发进度，还能确保项目的稳定性和可维护性。然而，考虑到未来的扩展性和用户需求的多样性，我们计划在后续开发中将系统逐步转化为插件形式。这种插件化的设计能够使系统更加灵活，便于集成到不同的平台和应用中，同时也为未来的功能扩展提供了便利。

初始论文数据库的构建：

在项目初期，我们建立了一个初始的论文数据库。该数据库涵盖了多个专业方向的论文，确保了我们能够对不同领域的知识进行初步的覆盖。这些论文经过精心筛选和整理，形成了一个具有一定规模和多样性的数据集。通过这个初始数据库，我们能够为后续的模式训练和微调提供坚实的基础。

基于 LangChain 框架的模式微调：

在初始论文数据库的基础上，我们采用了 LangChain 框架对大模型进行第一步微调。LangChain 框架以其强大的自然语言处理能力和灵活的模式训练机制，成为了我们微调过程的核心工具。通过这一步骤，我们能够使大模型初步具备对论文内容的理解和生成能力，从而为后续的用户交互打下基础。

模式部署与用户交互：

在完成第一步微调后，我们将大模型部署到网页平台上，开始与用户进行交互。用户可以通过网页界面与模型进行对话，上传自己的论文，并通过语义分析获得关键词标注。这些关键词不仅帮助用户更好地理解论文内容，还为后续的知识库扩展提供了数据支持。

外接知识库的构建与第二步微调：

在与用户交互的过程中，我们收集了用户上传的论文及其标注的关键词。通过关键词检索，我们能够从用户上传的论文中提取出与特定关键词相关的专业解释。这些解释不仅更加贴合论文的实际内容，还能够作为外接知识库的一部分，进一步丰富我们的数据库。随后，我们利用这些外接知识库对大模型进行第二步微调，从而有效增强模型对特定论文的解释能力。

用户笔记功能的实现：

为了进一步提升用户体验，我们开发了用户笔记功能。该功能允许用户针对论文中的某个词语或句子进行笔记记录。通过为每个句子或词语赋予动态 ID，并与笔记 ID 进行对应，我们实现了笔记与论文内容的精准关联。用户可以在阅读论文时随时查看和编辑自己的笔记，从而更好地理解和记忆论文中的重要内容。

参考文献的整合与管理：

在论文阅读过程中，参考文献的理解和关联性分析是非常重要的。为此，我们模仿了工业项目文件的格式，将论文中提到的参考文献进行整合，并存储到该论文的专属数据库中。当用户在阅读论文时遇到引用或参考的文献时，系统能够迅速展示这些文献之间的关联，帮助用户更好地理解论文的背景和上下文。

项目的系统架构如下：



五、技术实现：

1.关键词提取与检索

为了实现从论文文本中精准提取关键词，我们采用了基于深度学习的语义识别技术。具体实现步骤如下：

（1）文本预处理：首先对用户上传的论文进行预处理，包括分词、去除停用词、词性标注等操作。这些步骤为后续的语义分析提供了干净的输入数据。

语义嵌入：利用预训练的语言模型（如 BERT、GPT 等）对文本进行语义嵌入，将每个词语或短语转化为高维向量表示。这种向量化表示能够捕捉词语之间的语义关系。

关键词提取：通过计算文本中每个词语或短语的重要性得分（如 TF-IDF、TextRank 等算法），结合语义嵌入的结果，筛选出与论文内容最相关的关键词。这些关键词通常是论文中的核心概念、专业术语或高频词汇。

关键词标注：将提取出的关键词在原文中进行标注，并为每个关键词生成唯一的 ID，便于后续的交互处理。

（2）关键词高亮与交互设计

提取出的关键词会在用户界面中以高亮形式显示，用户可以通过点击高亮关键词触发进一步的操作：

高亮显示：在前端页面中，利用 HTML 和 CSS 技术对关键词进行高亮处理。例如，通过标签为关键词添加特定的样式（如背景色、字体加粗等），使其在文本中醒目显示。

点击事件绑定：为每个高亮关键词绑定点击事件。当用户点击关键词时，前端会向后端发送请求，携带关键词 ID 和上下文信息。

（3）关键词解释生成

当用户点击高亮关键词时，系统会立即生成与该关键词相关的详细解释。具体实现如下：

上下文分析：后端接收到用户点击的关键词 ID 后，结合关键词所在的上下文段落，利用预训练的语言模型生成针对该关键词的解释。生成的解释不仅包含关键词的定义，还可能包括其在论文中的具体应用、相关背景知识等。

外接知识库查询：如果关键词在初始论文数据库或用户上传的论文数据库中存在相关解释，系统会优先从知识库中提取信息，确保解释的专业性和准确性。

动态生成与返回：生成的解释内容以 JSON 格式返回给前端，前端通过弹窗或侧边栏的形式展示给用户。

（4）关键词定向跳转

为了进一步提升用户体验，系统支持通过关键词进行定向跳转，直接导航到论文中与该关键词内容密切相关的部分。具体实现如下：

关键词定位：在论文文本中，每个关键词都被赋予一个动态 ID，并与其在文本中的位置信息（如段落编号、句子编号等）关联。当用户点击关键词时，系统会根据位置信息快速定位到相关段落。

平滑滚动：前端通过 JavaScript 实现页面的平滑滚动，将用户视角直接移动到关键词所在的段落，并在目标位置添加短暂的视觉反馈（如高亮闪烁），帮助用户快速聚焦。

上下文关联：如果关键词在论文中多次出现，系统会提供一个列表，展示所有相关段落的位置。用户可以选择跳转到任意一个段落，进一步探索关键词的不同应用场景。

2.RAG 微调

（1）外部知识库的构建

为了实现 RAG 技术，我们首先需要构建一个高度结构化的外部知识库，用于存储论文及其相关的知识点。具体步骤如下：

论文数据收集：从用户上传的论文以及初始论文数据库中提取内容，包括标题、摘要、正文、参考文献等。

知识点提取：利用自然语言处理技术（如命名实体识别、关键词提取、语义分析）从论文中提取核心知识点，包括专业术语、理论概念、实验方法等。

知识结构化：将提取的知识点以结构化的形式存储到外部数据库中。每个知识点包括以下字段：知识点 ID、知识点内容、来源论文、上下文信息、和语义向量：（通过预训练模型生成的向量表示，用于后续的语义检索）。

数据库选择：我们选择使用向 MongoDB Atlas 来存储知识点的语义向量，以便支持高效的语义检索。

（2）检索模块的实现

RAG 技术的核心之一是检索模块，它负责从外部知识库中查找与用户问题相关的内容。具体实现如下：

用户问题向量化：当用户提出问题时，系统利用预训练的语言模型将问题转化为高维向量表示。

语义检索：

将用户问题的向量与外部知识库中的知识点向量进行相似度计算，根据相似度得分，从知识库中检索出与问题最相关的知识点及其上下文信息。

Top-K 结果筛选：系统会返回相似度最高的 Top-K 个知识点，作为生成模块的输入。

（3）生成模块的实现

在检索到相关知识点后，RAG 技术将这些知识点作为上下文输入到大语言模型中，生成更准确、更专业的回答。具体实现如下：

上下文拼接：将检索到的知识点与用户问题拼接成一个完整的输入序列。

模型生成：将拼接后的输入序列输入到大语言模型中，生成最终的回答。模型会基于检索到的知识点和自身的语言理解能力，生成既专业又易于理解的回答。

输出优化：对生成的回答进行后处理，包括去重、语法修正、格式优化等，确保回答的流畅性和可读性。

（4）模型再训练与微调

为了进一步提升模型在特定领域的表现，我们对大语言模型进行了再训练和微调：

训练数据准备：将外部知识库中的知识点与用户问题-答案对结合，构建训练数据集。

微调过程：利用构建的训练数据集，对预训练的大语言模型进行微调。微调的目标是使模型能够更好地利用外部知识库中的信息，生成更准确、更相关的回答。

评估与迭代：通过人工评估和自动化指标对模型的生成结果进行评估，并根据评估结果不断优化模型和检索模块。